PROGRAMMAZIONE

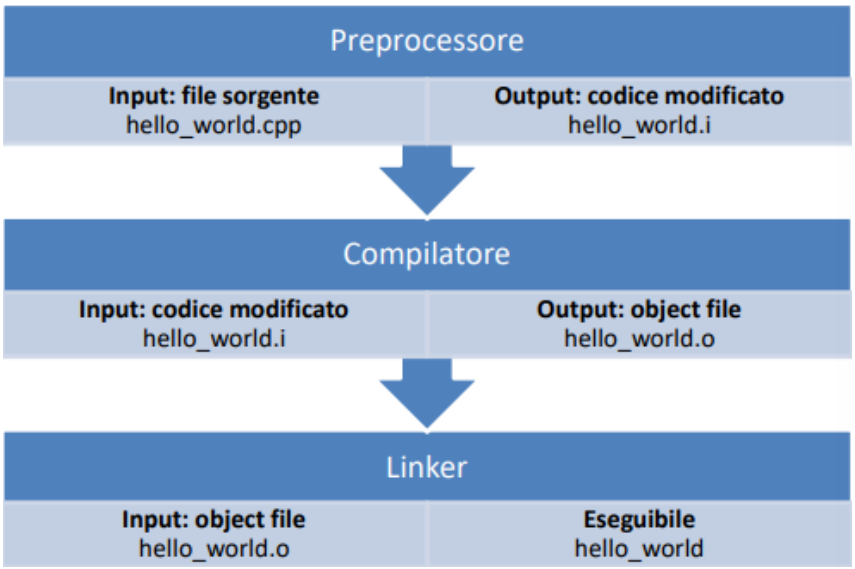
Intro **3**
Variabili **3**
Scrivere su una variabile 3
Espressioni e operazioni **4**
Istruzioni, Dichiarazioni & definizioni **5**
Dichiarazione 5
Definizione 5
Scope **5**
Librerie **6**
LINKING 6
Funzioni **7**
reference 7
chiamata a funzione 8
Classi **10**
struct 11
Costruttore 11
Overloading degli operatori **12**
puntatori **12**
void* e cast 13
Array stile C **14**
std::vector **14**
Aggiunta elementi 15
accesso al vettore 15
Free Store - allocazione dinamica della memoria **15**
new & free store 16
puntatore 16
Garbage e Memory Leak 17
Liberare la memoria **17**
Distruttore 19
Funzioni **20**
Funzioni membro const 20
Funzioni membro & helper function 20
costruttori e valori di default **21**
Initializer List **22**
Copia di oggetti **22**
Shallow copy 22
deep copy 23
move - spostamento di oggetti **24**
move constructor 24
move assignment 25
Puntatore this **25**

Template.....25
interi come parametri template.....27
header e template.....27
Polimorfismo.....28
Polimorfismo dinamico.....28
Polimorfismo statico.....30
Ereditarietà.....30
Is a.....32
has a.....32
Funzioni virtuali.....32
LAYOUT DEGLI OGGETTI.....34
Casting.....35
Dynamic cast.....35
Standard Template Library.....36
Contenitori.....36
Linked List.....38
Std::vector.....41
Std::string.....41
Std::list.....41
CMake.....41
sistema di building standard.....42
Eccezione.....44
Risorse ed eccezioni.....45
Smart Pointer.....47
unique_ptr.....48
Share_ptr.....48

[1.2]

Intro

C++ è un linguaggio **compilato:** (=compilazione è la traduzione del codice sorgente in codice macchina)
→ segnala errori (NO correzione)



comando: g++ <codice sorgente>.cpp -o <nome eseguibile>
(<nome eseguibile> è il file di output prodotto)

[2.1]

Variabili

variabile: è un **oggetto** con un proprio **nome** (necessario per poter accedere all'oggetto) che ha uno specifico **tipo**

- **oggetto =** il "luogo"/regione di memoria dove è contenuto il dato
- **tipo =** definisce la tipologia di contenuto
  - range di valori (+ dim in byte)
  - operazioni possibili

Scrivere su una variabile

Assegnamento
con "=" (*es. int a = 2*)

type safety: controllo sulle conversioni tra variabili (trasferire dati da una variabile all'altra)

- se è **type safe** i dati preservano il loro valore o hanno una buona approssimazione (*es verso una capacità maggiore)*
  (quando un oggetto è usato solo seguendo le regole del suo type)
    - *ERRORE: usare variabili non inizializzate*
- **narrowing:** valore convertito in qualcosa di diverso dal valore iniziale (ERRORE-LOGICO)

Inizializzazione
→ è stata introdotta la notazione **{}** che evita le narrowing conversion (altrimenti non vengono segnalati errori)
SI effetto su memoria
double x {2.7}

Dichiarazione
dichiarazione che da un nome ad un oggetto.
NO effetti su memoria

Definizione
dichiarazione che setta la memoria per un oggetto

[2.2]
Espressioni e operazioni

espressione: calcola un risultato a partire da uno o più operandi

- **lvalue** (left value): elemento a sinistra di un operatore di assegnamento

espressioni costanti = named object sul quale **non** si può ridefinire una nuova regola dopo che è stata inizializzata
*es **constexpr** double pi = 3.14159*

- ***constexpr**: valore noto a tempo di compilazione*
- ***const**: valore noto solo a tempo di esecuzione (assegnato a inizializzazione)*

side effect: operatori che modificano gli operandi su cui operano (es ++). La libertà nell'**ordine** delle operazioni permette al compilatore di ottimizzare
ATTENZIONE: usare la stessa variabile soggetta a side effect è pericoloso (*non so in che ordine vengono effettuate le operazioni → non so il risultato).*
es. v[i] = ++i

OPERATORI >> e <<

sono **sensibili al tipo:** funzionamento dipende dall'operatore.

- << inserisce l'operando in uno stream
- >> estrae un operando da uno stream

Trasformano: testuale ↔ rappresentazione interna della variabile

[2.3]

Istruzioni, Dichiarazioni & definizioni

Istruzione

> **Istruzione: una parte di codice che specifica un'azione.** Termina con ";", che è necessario per interpretare il codice.

Non sono istruzioni le direttive del preprocessore (es: #include).

Alcuni tipi di istruzioni sono: expression statements, dichiarazioni, selezione (if, switch), iterazione (while, for), ...

Dichiarazione

> **dichiarazione: introduce un nome in uno scope:** specifica il nome, il tipo e (a volte) un valore iniziale.
> (*variabili, funzioni,...*)

Non si dice nulla sulla "riserva" della memoria

es. double a;

int func();

Esistono perché può servire creare delle entità **prima di sapere la definizione completa** (es file multipli)

-> permette di **riferirsi** ad un'identità

-> raccolte in un **header** file → permettono di **condividere** funzioni

Definizione

> **Definizione :** dichiarazione che specifica **completamente** l'entità dichiarata.
> Tutte le info per creare l'entità

- variabile: nome, tipo e **riserva la memoria**
- Funzione: corpo della funzione
- Classe: variabili e funzioni membro

es: int func(){ return 2};

[3.1]
Scope

=**regione di testo.** Un name è dichiarato in uno scope ed è valido al suo interno *(dalla dichiarazione alla fine dello scope)*

Tipi:

- **globale:** area a di fuori di ogni altro scope
    - Variabili: problematico → diventa accessibile in tutto il file sorgente, ogni funzione ne può modificare il valore
    - funzioni: spesso hanno scope globale
- **di classe:** all'interno di una classe
- **locale:** nel blocco {} o nella lista di argomenti di una funzione
- **statement:** es for
- **namespace:** un'area (con nome) annidata all'interno dello scope globale (o altro namespace)
    - Permettono di organizzare classi, funzioni, dati e tipi dentro a uno spazio di nomi **senza definire un tipo**
    - **using directive:** associa un namespace a un nome (solo con namespace noti come std (*using namespace std; using std::string)*

[3.2]
Librerie

ATTENZIONE: direttive (*es. #include)* NON sono istruzioni
Direttive: per importare gli identificatori di altre entità

> **file header:** meccanismo per **importare** funzioni/classi/variabili scritte da altri (librerie)
> = insieme di dichiarazioni di entità utilizzabili dove viene incluso tale file

- header di **sistema**: inclusi con <> (*#include <iostream>)*
- header definiti dall'**utente**: con ""

Un file sorgente può includere più header

***osservazione**: in grandi progetti si inseriscono le definizioni e dichiarazioni in file sorgente e in file header (ordinate e utilizzabili da altri file)*

Devono essere incluse le **include guards**

```
#ifndef RATIONAL_H
#define RATIONAL_H
// ...
#endif // RATIONAL_H
```

include guards: assicurano che il compilatore processi il file **una sola volta** (indipendentemente da quante volte viene inclusa). Sono una serie di direttive del processore

- **librerie dinamiche:** una copia sola della libreria per più object file
    - in fase di **esecuzione**
- **librerie statiche:** libreria ha la sua classe object
    - gli eseguibili creati possono venire **separati**
    - durante la fase di **compilazione**
    - ***vantaggio***: tutte le funzionalità vengono incluse nel file finale e usate senza alcuna dipendenza
    - ***svantaggio:*** ogni aggiornamento richiede di ricompilare tutti i programmi che la usano

LINKING

un programma normalmente consiste in più parti spesso scritte da più programmatori. Il **linker collega** le parti di codice diverse necessarie per creare l'eseguibile

[4.1]

Funzioni

> **funzione:** una *named* sequenza di *statement.* può ritornare un valore (**return value)**

```
int square(int x){
    ....
}
```

*prima riga = **definizione** (nome, parametri e tipo del return value)*

parametri/argomenti

- si **devono** dare alla funzione i parametri come sono richiesti

- il passaggio avviene per **valore** [vedi CODICI]
- **variabili locali** inizializzati ad ogni chiamata

=> problemi problemi 🚗

[4.2]
reference
se si usa un **vector** o un altro argomento di dimensioni elevate, la copia potrebbe essere **inefficiente**

=> non ha una locazione di **memoria propria,** ma punta a quella di una variabile ordinaria già esistente (*equivale a dare un nome alternativo ad una variabile già esistente*)

es. void print(vector v)
- uso la **const reference** attraverso: *es. void print(**const** vector**&** v)*
  - ***const**: print non può modificare il vettore [NON obbligatorio, solo se si deve garantire l'immutabilità]*
  - ***&**: reference → indirizzo dove "andare a vedere i risultati"*
  - *accetta anche non lvalue (es. anche valori)*

cosa fare se una funzione deve modificare un vector passato per parametro ?
Uso: **pass-by-reference**: *es. void print(vector**&** v)*
ATTENZIONE: **senza** la reference si modificano i valori solo nella funzione ma non al di fuori di essa (è una copia)

=> **accetta solo** lvalue

altri usi:

```
int i = 7;
int& r = i;     // reference a i
r = 9;          // i ora contiene 9
i = 10;         // i ora contiene 10
cout << r << ' ' << i << '\n';
```

output: 10 10

riepilogo:
- **pass-by-value:** oggetti piccoli
- **pass-by-const-reference**: oggetti grandi non modificabili

- **pass-by-reference**: solo se/quando necessario
    - manipolare contenitori
    - produrre più output
    - modifiche all'oggetto passato
- ritorna un risultato piuttosto che modificare un oggetto attraverso la reference (es. x = incr(x) è più leggibile di incr(x))

check e conversioni

I valori passati vengono convertiti nel tipo del parametro

```
void f(double x);
void g(int y){
   f(y) //OK! y viene convertito da int a double
}
```

ATTENZIONE: è poco leggibile e potrebbe produrre un risultato non atteso (es → passaggio da double a int)
=> meglio una **conversione esplicita**

[4.3]

chiamata a funzione

=> provoca la creazione di una struttura: **Record di attivazione** che contiene:

- copia dei **parametri**
- **variabile locale**

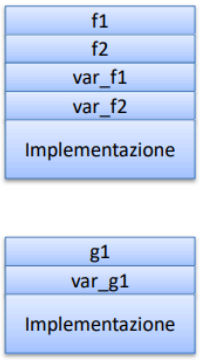
Sono impilati in una zona di memoria: **stack**
Il tempo di inserimento è costante

libro: ogni funzione ha la sua struttura che contiene i dettagli di layout del suo record di attivazione

- informazioni e valori che deve ritornare alla funzione chiamante

```
int f(int f1, int f2)
{
    int var_f1;
    int var_f2;
    // ...
}

int g(int g1)
{
    double var_g1;
    // ...
}
```



se chiamo f(x) e poi g(x) le info di f vanno sotto

LAYOUT DI MEMORIA

1. **text segment:** contiene le istruzioni eseguibili
2. **initialized data segment:** variabili globali e statiche
3. **uninitialised data segment:** variabili globali statiche non inizializzate esplicitamente
4. **stack:** LIFO
5. **heap:** gestito tramite **new** e **delete** → allocazione dinamica della memoria

VARIABILI

Suddivise per

- **durata**
- **scope**

Variabili statiche	• inizializzate una volta • mantengono valore anche fuori dallo scope • **globali** (sempre) **o locali** (se dichiarate static)
variabili globali	• inizializzate prima del main fino alla fine del programma • **statiche**
variabili locali automatiche	es inizializzate ad ogni chiamata di funzione o nel loop

(da simulazione → potrebbe essere sbagliato)

es. se definisco una variabile statica (es. static int = 9) dentro una funzione

- *viene inizializzata solo la prima volta che viene invocata la funzione*
- *mantiene il valore tra una chiamata e la sua successiva*

- *contenuto nell initialized list (NO STACK)*

[5.1]

Classi

operazioni essenziali:
- ➔ costruttore con elementi
- ➔ costruttore di default
- ➔ costruttore copia e assegnamento (definirli o proibirli)
- ➔ costruttore move e assegnamento (definirli o proibirli)
- ➔ distruttore (solo se acquisisco risorse)

- Tipi built-in
  - int
  - char
  - double
- Tipi **definiti dall'utente** / User Defined Types (UDT)
  - Classi definite in std
    - string
    - vector
    - ostream
  - **Classi** definite dal programmatore
  - **Struct** definite dal programmatore

Elementi dei tipi:
1. rappresentazione: un tipo deve sapere i dati per un oggetto (**stato**)
2. operazioni: tipo deve sapere le operazioni applicabili all'oggetto

classe: specifica
- come sono rappresentati gli oggetti
- come possono essere **creati, usati, distrutti**

Entità per definire la classe = **membri (***usati per definire la classe***)**
- **data member:** definisce la rappresentazione di un oggetto della classe. Privati di default
- **funzione membro:** fornisce le operazioni possibili sull'oggetto e operano sul suo **stato**. **Dichiarate** dentro la classe e **definite** dentro e fuori. Chiamate con oggetto.funzione()

- **N.B.** funzioni definite nella classe (**inline**) → modifiche richiedono la ricompilazione di tutte le classi che la usano

Helper function: eseguono le operazioni più comuni, spesso a supporto della classe. Definite **fuori** dalla classe e chiamate come normali funzioni

accesso ai membri

- oggetto.membro
- funzioni membro: garantiscono che i dati rimangano **coerenti**

Interfaccia	implementazione
public, accessibile all'utente	**private**, non direttamente accessibile e lettura e scrittura solo tramite interfaccia

[5.2]

buona pratica:

- *funzioni → unico modo per accedere ai dati*
- *proteggere i dati membro (evitare invalidazione dell'oggetto)*
- *NON riferirsi direttamente a un membro privato → OK attraverso una funzione pubblica*
- *non definire funzioni membro nella definizione della classe*
- *non entrare mai in uno stato **non** valido (usare eccezioni ecc)*

funzioni membro:

segnalare errori

es. classe Invalid → classe dedicata per creare un tipo che segnala uno specifico errore

eccezione: meccanismo che separa il rilevamento di un errore dalla sua gestione

– **Rilevamento**: spesso in una funzione chiamata

– **Gestione**: spesso nella funzione chiamante

unico modo per far fallire un costruttore

struct

membri sono **pubblici** di **default**

quando non ci sono funzioni e i membri possono assumere qualsiasi valore (non hanno **invariante**)

Costruttore

costruttore: funzione membro per inizializzare gli oggetti della classe con dati validi (di default o con controllo sulla validità dei parametri passati)

ERRORE SEGNALATO DAL COMPILATORE: NON inizializzare un oggetto attraverso il costruttore

init list

```
Date::Date(int yy, int mm, int dd)
    : y{yy}, m{mm}, d{dd}
{}
```

[5.5]

Enumerazioni

enumerazioni: tipo definito dall'utente che specifica set di valori.
=associa un valore numerico a ciascun elemento elencato nell'enumerazione

con **enum class**: crea uno scope

```
enum class Month{
    jan = 1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec
};
```

Mi riferisco a jan come **Month::jan**

[6.1]

Overloading degli operatori

= implementare operatori come **operandi di tipo definito dall'utente**
tramite: helper function, funzioni membro.
Ha nome specifico: *es. operator++*
Deve avere significato intuitivo

implementato con **helper function**	operator+, operator-, operator*, operator/ → due argomenti (**almeno uno UDT**, l'altro

	può essere anche built-in)
implementato con **funzioni membro**	il primo argomento (UDT) viene sostituito con **l'oggetto** su cui viene **invocata** la funzione
<< e >>	• **operator <<**: un argomento ostream&, ritorna ostream& • **operator>>**: un argomento istream&, ritorna istream&

[6.2]

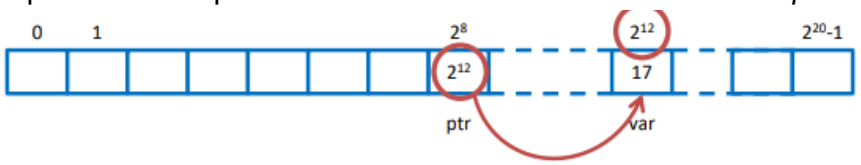
puntatori

puntatore: particolare tipo di variabile che contiene un indirizzo di memoria, utile per poter accedere al dato puntato

da YouTube
variabili che permettono di salvare un **indirizzo di memoria** ad un'altra variabile.
Servono per lavorare con la locazione dinamica di memoria, cioè durante il programma viene allocata della memoria (per variabili ecc) e uso i pointer per **cambiare** e **accedere** ai dati salvati in questa memoria (durante il programma)

punta ad un tipo (noto) situato all'indirizzo di memoria contenuto.
definizione: <tipo>* <nome_puntatore>

E' possibile acquisire l'indirizzo di una variabile: *es. int* ptr = &var*



dereference: per "vedere" il contenuto della memoria puntata
*cout<< *p*
→ *restituisce una **lvalue***

[6.5]

void* e cast

è un puntatore (a memoria raw) che salta ogni tipo di controllo. Concetto puro di indirizzo di memoria senza istruzioni su come usarla

```
int i;
void* pv1 = &i;
```

	puntatori	reference
assegnamento	cambia il valore del puntatore e non del valore contenuto (altrimenti usa la dereference)	cambia il valore
spostare	è possibili spostare il puntatore assegnando un nuovo indirizzo	Non si può più spostare dopo l'inizializzazione
accedere	* oppure []	nessun operatore necessario
copy	due riferimenti allo stesso oggetto (**shallow copy**)	copio il contenuto (due oggetti dello stesso valore -> **deep copy**)
non valido	nullptr	non esiste

cosa usare?

- *funzione che **ritorna** valore: meglio di tutti, chiaro. NO quando gli oggetti sono grandi e **non** hanno il **move constructor***
- usare puntatori: se si deve gestire il caso nullptr
- usare reference: altri casi

[6.4]

Array stile C

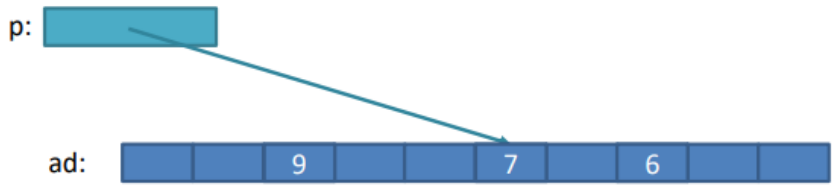
Array: sequenza omogenea di oggetti dello stesso tipo allocati in spazi di memoria contigui

- accesso casuale
- nessun controllo nella lettura e scrittura fuori range
- **limitazione** sulla dimensione → conosciuta a tempo di compilazione

Se passati come **argomenti** nelle funzioni vengono trasformati in **puntatori**

E' possibile usare l'**aritmetica dei puntatori** per scorrere un array

```
double ad[10]
double *p = &ad[5]
*p = 7;
p[2] = 6;
p[-3] = 9;
```



aritmetica dei puntatori: somma o sottrazione di **n** da un puntatore → sposta il puntatore di n slot a destra o a sinistra
=> **dimensione** dello slot: dipende dal dato puntato

Nome di un array è un puntatore const al suo primo elemento ed è un rvalue (valore e non una variabile)
=> non è possibile usarlo come copia

std::vector

= sequenza **dinamica** di oggetti uguali

- accessibili con [] o con verifica
- aggiunta di elementi: **push_back**()
- numero di elementi: **size**()

Comportamento implementato nella libreria standard non dall'hw

```
vector<int> v  = {5, 6, 9}      //creazione
v[1] = 4;      //accesso
```

Aggiunta elementi

attraverso: push_back()

- push_back() è una **funzione membro** della classe Vector (come size())

- **struttura dinamica** → non hanno una dimensione fissa, **cambia dimensione** all'aggiunta di ogni elemento
    - rialloca ad ogni aggiunta ⇔ array
- dimensione è nota a **tempo di esecuzione** (*posso leggere la dim a standard input*)

=> c++ è un linguaggio ad alto livello

[9.1]

accesso al vettore

Per la lettura e la scrittura in un vector uso **l'operator []**

es. lettura

```
public:
    double& operator[] (int n) {
        return elem[n];
    }

    double operator[] (int n) const{
        return elem[n];
    }
```

- **&**: altrimenti non è possibile

```
double x = v[2];
v[3] = x;            //v[3] non è un lvalue
```

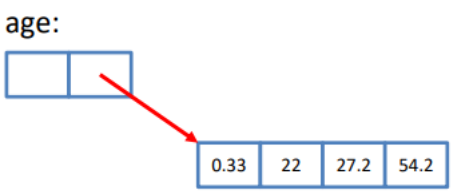
- **const**: per poter ritornare una copia o una const reference. Però ritorna una reference a membro privato. Giusto? → dibattito su stackoverflow

[7.1]

Free Store - allocazione dinamica della memoria

Zona di memoria per creare strutture di dimensione dinamica (*come vector*)

es. implementare vector

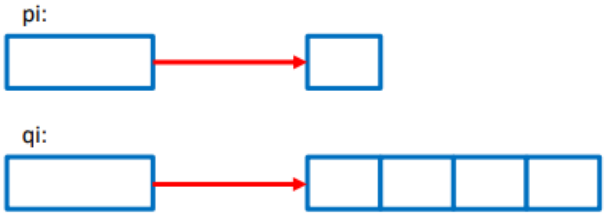


come si implementa questa struttura?
slide 07_1

- Per accedere al free store uso **new** e i **puntatori**

new & free store

```
double* pi = new double;
double* qi = new double[4];
```



- restituisce **un puntatore**:
  - All'oggetto allocato (se singolo)
  - Al **primo elemento** (se array)
- Il puntatore **non sa** se sta riferendo ad un oggetto singolo o ad un array (programmatore deve sapere per poterlo gestire)

puntatore

- è una **variabile**
- mantiene l'indirizzo
- ATTENZIONE: se perdo l'indirizzo non sono più in grado di raggiungere il blocco nel free store → il blocco resta occupato, ma inutilizzabile→ **memory leak**.
  - *es.* q = p; → sovrascrivo q, che viene perso (memoria inutilizzabile, **garbage**)
- posso usare l'**aritmetica dei puntatori** e [] (→ per vedere la memoria come array)
  - p++ permette di passare alla cella/slot successiva nella struttura -> indipendentemente dalla dimensione della cella
  - **Nessun controllo sui limiti (range check)** → *ERRORE DI ESECUZIONE*
  - new non inizializza i tipi built-in.

INIZIALIZZAZIONE DEL PUNTATORE

```
double* p0;  NO!!!
double* p1 = new double;  circa
double* p3 = new double {5.5}
double* p5 = new double[]{0,1,2,3,4}; // dimensione dedotta

con UDT
X* px1 = new X;       // ok se esiste costruttore di default
```

```
Y* py3 = new Y{13};    // ok se costruttore accetta int
Y* py4 = new Y[17]{0,2,3,4,...}; // ok
```

- **NON** inizializzare un puntatore è **un errore grave:** non è inizializzato di default a *nullptr,* ma ad un indirizzo casuale → memoria non valida

- Buona pratica, controllare sempre: if (p0 != nullptr) { ... } → SE INIZIALIZZATO CORRETTAMENTE

Garbage e Memory Leak

- **Garbage** = zona di memoria **allocata ma non raggiungibile, si perdono i riferimenti**.
    - vengono impiegati molti cicli di clock per eliminare questa zona
- **Memory leak** = perdita del riferimento → spreco di memoria
    - Può portare a **esaurimento risorse**.

Liberare la memoria

In **C++** la memoria **non viene liberata automaticamente** → **nessun garbage collector** → Deve liberarla il **programmatore**

- La produzione di *garbage* è un **errore tecnico grave**.

Perché liberare la memoria manualmente?
- **Efficienza** → il garbage collector richiede molte risorse (molti cicli di clock) per analizzare la memoria.
- **Controllo** → il programmatore decide *quando* liberare la memoria, rendendola subito riutilizzabile.

deallocazione

- `delete` → per oggetti allocati con `new`
- `delete[]` → per array allocati con `new[]`

Entrambe si applicano ai **puntatori**.

Dopo delete, l'area di memoria è *libera*, ma il puntatore mantiene l'indirizzo → **non usarlo più** se non è stato riassegnato o impostato a nullptr.

Il **sistema operativo** (è il SO a gestire il free store) può riutilizzare quell'area: manipolarla è **scorretto**.

```
double* calc(int res_size, int max) {
   double* p = new double[max];
   double* res = new double[res_size];
   delete[] p;        // libera la memoria
   return res;
}

// ...
double* r = calc(100, 1000);
delete[] r;
```

res viene copiato in r per cui non viene perso il contenuto (indirizzo)

- Un oggetto allocato dinamicamente creato in uno **scope** può essere passato in un **altro scope**
    - es. res e r: viene copiato solo il puntatore, la memoria rimane la stessa
- Il **chiamante** (dove chiamo calc) diventa responsabile della **deallocazione**.

Errori frequenti

- Doppia cancellazione
    - Dopo la prima delete, il **puntatore non è più proprietario** della memoria.
    - Potrebbe puntare a **memoria già riutilizzata** dal sistema operativo.

  Soluzione: dopo delete, impostare sempre a `nullptr`.

- Dangling pointer (puntatore pendente)
    - Puntatore a memoria **non più valida** (liberata o non inizializzata).

  soluzione: settare il puntatore a nullptr

Distruttore

A fine vita di un oggetto, serve un meccanismo **automatico** di pulizia => **Distruttore** (pk clean_up() è gestita dal programmatore, non ci si può permettere di avere il dubbio se è stato eliminato o no)

- chiamato automaticamente dal compilatore quando:

    - L'oggetto **esce dallo scope**;
    - L'oggetto viene **deallocato**.
- Se non definito, il **compilatore genera un distruttore di default**.
    - ATTENZIONE: non dealloca su zone di memoria allocate manualmente da noi
- possibile definire un **distruttore esplicitamente** nella classe che stiamo definendo
    - Se inserisco **delete** → dealloca automaticamente

ATTENZIONE:

➔ Gli oggetti automatici (come v) vengono **deallocati automaticamente**

➔ Gli oggetti sul **free store** (come p) → devono essere deallocati **manualmente**.

Pattern di gestione risorse

- Un oggetto acquisce risorse nel **costruttore**
- vita dell'oggetto
    - acquisisce altre risorse
    - libera alcune risorse
- Rilascio nel **distruttore**

Operatore	Azioni
`new`	Alloca memoria + chiama costruttore
`delete`	Chiama distruttore + libera memoria

Pro e contro dell'allocazione dinamica

Vantaggi

- Possibilità di gestire **grandi quantità di dati**.
- Dimensione **decisa a runtime**.
- **Controllo preciso** del ciclo di vita.

Svantaggi

- Necessità di **gestione esplicita** della memoria.
- Rischio di **errori** (leak, dangling, doppia delete).
- **Overhead computazionale** (interventi del sistema operativo).

Usare allocazione dinamica **solo se necessari altrimenti** → **errore di design**.

[8.1]

Funzioni

Funzioni membro const

Permette di non modificare un oggetto quando la si chiama su di esso

```
Class Date {
public:
      int day() const;
      ....
}

void some_function( Date& d, const Date& start_of_term) {
   int b = start_of_term.day();
}
```

É un errore modificare un membro in una funzione const => errore di compilazione

Es. In Date :: day() const non posso fare ++d

Funzioni membro & helper function

Una non-member funzione semplice ed efficiente deve essere implementata **fuori** dalla classe => in questo modo solo le funzioni membro della classe devono essere controllate in caso di corruzione di dati.
Consiglio: implementare più funzioni fuori dalla classe possibili

Helper function	
funzioni membro	permette la **protezione** dei dati membro

	è consigliabile siano poche *es. operator== e operator!=*

[8.2]

costruttori e valori di default

In un'interfaccia **DEVONO** essere forniti i costruttori.

Per alcuni oggetti/dati esistono dei **valori di default** (es. La stringa vuota).
È buona pratica implementare un **costruttore di default** oltre ai costruttori che permettono di creare un oggetto con dei valori di default

- Per i tipi **built-in**: inizializzo a **0**
- **Vector**: vettore vuoto
- **Stringa**: stringa vuota
- **Oggetti**: con valori "sensati" per tale oggetto (es. Date → una data specifica)

```
string s1;
vector<string> v1
int{};
double{};
```

Per inizializzare ai valori di default:

- costruttore di default
- **in-class initializers:** nei membri

```
int y {2001}
   oppure
Date(int y = 2001, Month m = Month::jan, int d = 1)
//implemento in una volta sola
    - costruttore di default
    - normale costruttore
    - costruttore specificando solo alcuni membri (ATTENZIONE
      all'ordine)
//SOLO se fornisco il valore di default di tutti i parametri
```

[8.3]

Initializer List

E' possibile inizializzare la classe Vector nel seguente modo:
vector v1 = {1.2, 7.89, 12.34}
invece che creare il vettore e inserire i valori con push_back()
=> fornisce un oggetto della **std** di tipo **initializer_list<T>**

Nella classe Vector → Costruttore

```
vector(std::initializer_list<double> lst)
        : sz{lst.size()}, elem{new double[sz]}
{
        copy(lst.begin(), lst.end(), elem);
}
```

copy: copia una sequenza di elementi (di ogni tipo) delimitata dai primi **due** argomenti (ultimo indica la prima posizione libera) e puntata dal **terzo** parametro

osservazione: initializer_list è passato per copia

[8.4]

Copia di oggetti

Costruttore copia di **default**
- fa una **shallow copy** → copia membro a membro
- copia **tutti** i membri

ATTENZIONE: E' sempre necessario definire la copia oppure indicare che la copia è proibita → altrimenti è il compilatore ad operare spesso in modo scorretto
Se uso una classe già esistente **DEVO** sapere **come** gestisce la copia

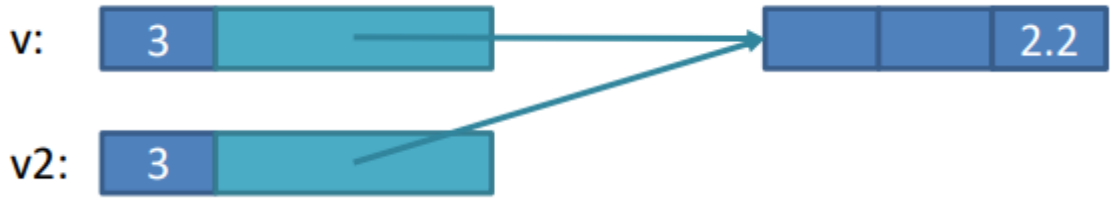
analizzo con vector
Shallow copy

copia dei **puntatori** o **reference,** senza copiare i dati

```
vector v(3);
v.set(2, 2.2)
```

```
vector v2  = v;
```

I due vettori puntano allo **stesso elemento** in memoria



Problematiche

- **doppia deallocazione**: all'uscita dello scope viene chiamato sia il distruttore di v che di v2 → deallocazione della stessa area di memoria
- se viene **modificata la dimensione** di uno, non viene modificata la dimensione dell'altro (si "disallineano")

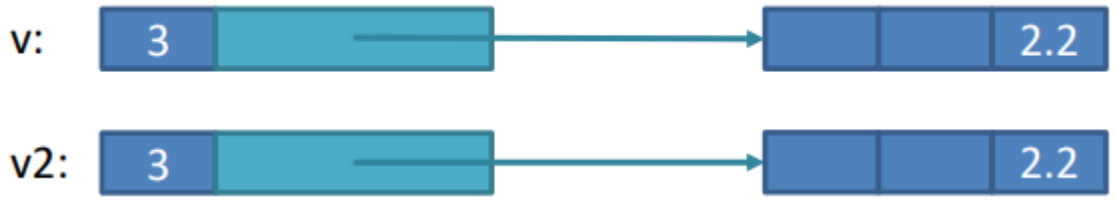
deep copy

copia i **dati** e vengono definiti **costruttore di copia** e **assegnamento di copia**

costruttore di copia

```
vector(const vector& arg)
        : sz{arg.sz}, elem{new double[arg.sz]}
{
        copy(arg.elen, arg.elem+sz , elem);
}

vector v2 = v; // vector v2 {v};
```



È un costruttore di copia che accetta una reference costante a un oggetto della stessa classe

assegnamento di copia

```
vector& vector =(const vector& a)
{
        double *p = new double[a.sz];     //nuova copia
        copy(a.elem, a.elem+a.sz , p);
```

```
        delete[] elem;                          //elementi vecchi
        elem = p;                               //punto ai nuovi dati
        sz = a.sz;

        return *this;
}

vector v2 = v;
```

gestisce i dati vecchi

[8.5]

move - spostamento di oggetti

= modo per **spostare** tutti i dati (per es. da un vettore), **invalidando** la sorgente
es. è l'alternativa più elegante e corretta
*

```
vector fill( istream& is){
    vector res;
    for ( double x; is >> x; ) res.push_back(x);
    return res
}

void use{
    vector vec = fill(cin)
}
```

res si distrugge, ma lasciando mettere i dati in vec (che si sta creando). Se ho tanti dati dentro res si copiano tutti dentro vec

move constructor

```
vetor::vector( vector&& a)
      :sz{a.sz}, elem{a.elem}{
     a.sz = 0;
     a.elem = nullptr;
}
```

- accetta una **rvalue (**con **&&)** e ritorna una **reference**

- **no const:** perché invalido la sorgente (la sto modificando)

In questo modo, in * quando fill ritorna res il compilatore può usare il **move** invece che la copia

ATTENZIONE: programmatore implementa il move constructor ma è il **compilatore** a scegliere tra move e copia, compilatore rileva le **condizioni**

move assignment

```
vector& vector::operator=( vector&& a)
{
        delete[] elem;
        elem = a.elem;
        sz = a.sz;
        a.elem = nullptr;
        a.sz = 0;

        return *this;
}
```

[9.2]

Puntatore this

- **=puntatore all'oggetto stesso**
- parametro **implicito** delle funzioni membro (generato automaticamente)
- è **immutabile**

per es. return *this (nell'assegnamento di copia) ritorna una **self-reference** all'oggetto stesso
ATTENZIONE: sto ritornando l'oggetto **derefenziato**

può essere usato per **accedere all'oggetto corrente** da una **funzione membro**

```
int vector::size(void){
    return this->sz            //non strettamente necessario (questione di stile)
}
```

[9.3]

Template

> =permette al programmatore di usare un tipo come **parametro** per una classe o una funzione
> *es. vector<double>*

Compilatore genera codice necessario per **ogni tipo** per cui il template è specializzato → tempi di compilazione molto lunghi

→ "genera funzioni come push_back con un **tipo generale T non ancora ben definito,** poi nell'utilizzo posso usare push_back con vettori double, int ecc → specializzo"

class template	chiamato anche type template.
specialization (template instantiation)	• processo di **generazione** di una classe su un tipo **specializzato** • in **tempo di compilazione** • *specifico ciò che è stato lasciato generale*

```
template<typename T>
class vector{
    int sz;
    T* elem;
public:
    explicit vector(int s) : sz{s}, elem{new T[s]}{ ecc
```

→ explicit elimina la conversione automatica che solitamente si fa nei costruttori

-

```
template<typename T>
void vector<T>::push_back(const T& d)....
```

→ se per esempio sto specializzando con una string, il compilatore genera:
*void **vector<string>**::push_back(const **string**& d)....*

function template

```
template<typename T>
T myMax(T x, T y){
        return(x > y) ? x : y;
}
```

- uso i template per poter scrivere una volta sola, in generale, un'operazione che viene **implementata solitamente per molti tipi diversi**
- **l'operazione > deve essere definita** per i tipi che stiamo confrontando (o è built-in o devo definirla io) => altrimenti ERRORE DI COMPILAZIONE

interi come parametri template

=numero di elementi è noto a tempo di compilazione e non più modificabile

- se definisco un vettore di **m** elementi e uno di **n** elementi (con elementi dello stesso tipo) => i due vettori vengono considerati come **tipi diversi**
- **Non** è necessario occuparci della **riallocazione**

```
template<typename T, int N> struct array{
      T elem[N];

      T& operator[] (int n);
      .....
}
```

parametro noto a tempo di **compilazione**

vantaggi

- efficiente
- **non** necessario il **free store**
- può esprimere un concetto (es. array di 3 elementi → tripletta, o RGB)

svantaggi

- poca separazione tra dichiarazione e definizione
- errori di compilazione poco comprensibili
- il **compilatore** spesso richiede che i template siano **completamente definiti** prima di essere usati

[9.4]

header e template

per mantenere la separazione tra interfaccia e definizione uso un header **diviso** in due:

- **file.h:**
    - definizioni di classi
    - dichiarazione di funzioni

- **File.hpp**
    - definizioni di funzioni

combinati con **#include** "file.hpp" alla fine del file h
=> ora tutti i file usano .h devono essere compilati => tempi di compilazione molto lunghi

//file.hpp **#ifndef** vector_hpp **#define** vector_hpp --- **#endif**	//file.h **#ifndef** vector_h **#define** vector_h --- **#endif**

[9.5] e [12.2]

Polimorfismo

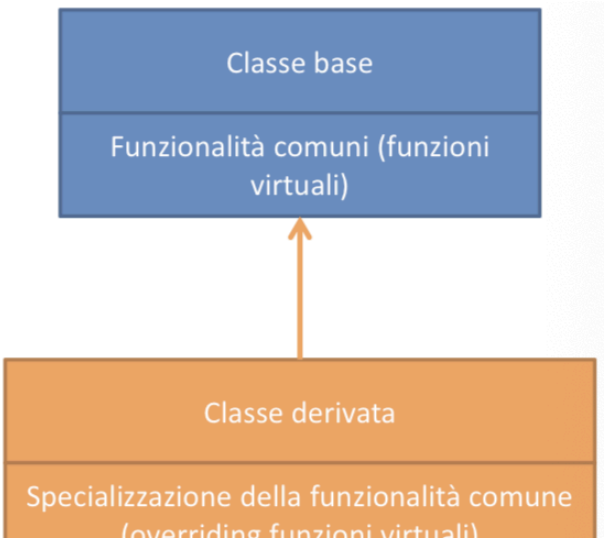
=multiforma
abilità di associare componenti specifici diversi in un'unica notazione
- **dinamico:** con ereditarietà (a run-time)
- **statico:** tramite template (a compile-time) - concetto di programmazione generica

programmazione generica = scrivere codice che funziona con **tipi diversi forniti successivamente** (devono soddisfare determinati requisiti sintattici e semantici)

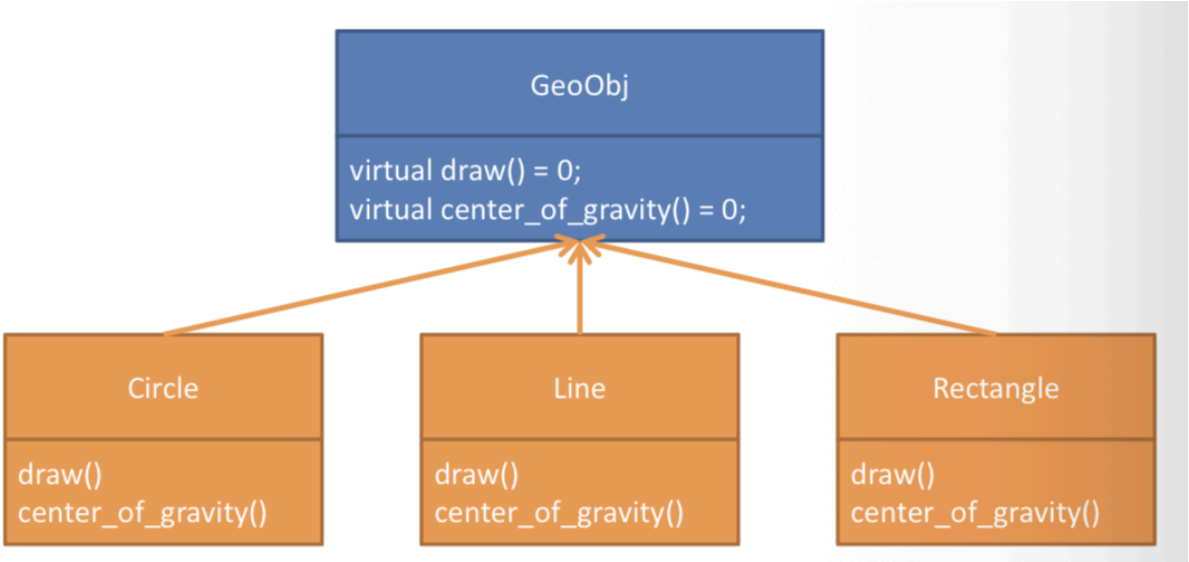
Tipi di polimorfismo
- **Overloading di funzioni:**
- **Overloading di operatori**
- Polimorfismo dinamico
- Polimorfismo statico

Polimorfismo dinamico

gestito tramite
- **Ereditarietà**
- **funzioni virtuali** —> *programmatore vuole implementare una classe ma non sa come (crea un'interfaccia e obbliga le altre funzioni a fare un'overloading delle funzioni definita ad essa). Le classi derivate son **obbligate** a fare l'override ("ridefinirle")*



Es.



Osservazione

```
Void MyDraw(GeoObj& obj)
    obj.draw();
```

Come obj posso passare Circle, Line, Rectangle -> NO GeoObj perché non si può creare un suo oggetto

Polimorfismo tramite **ereditarietà**

- = di tipo **Bounded** => interfaccia delle classi derivate è **vincolata** dalla classe base
- Effettuato tramite tempo di **esecuzione**

Polimorfismo statico

esempio: Circle, Rectangle, Line sono classi che implementano tutte le stesse funzioni (Non sono obbligate perché non c'è GeoObj e non ci sono le funzioni virtuali)

*Polimorfismo tramite i **template***

- Di tipo **unbounded**: interfaccia non è predeterminata/obbligata da altre classi base.
- Effettuato tramite tempo di **compilazione**

```
template<typename GeoObj>
Void MyDraw(GeoObj const& obj){
        obj.draw();
}
```

In questo caso le classi devono comunque implementare le funzioni anche se non obbligate da una funzione virtuale poiché altrimenti non riuscirei a fare un draw(Line) —> obbligo **implicito**
=> ci sono fino a tre funzioni MyDraw<Circle>(), MyDraw<Line>, MyDraw<Recangle>

Differenza con polimorfismo dinamico:

- **dinamico:** c'è una funzione e poi mi specializzo nelle varie classi derivate.
- **Statico**: c'è un unico nome della funzione, ma tre funzioni diverse. E' il compilatore a decidere quale usare. Il comportamento è predeterminato attraverso il compilatore (*come uso normale dei template)*

[11.1]

Ereditarietà

Si definiscono due tipi di classi:

- **classe base:** (classe astratta) dove è possibile raggruppare tutte le caratteristiche comuni. Implementa un interfaccia
    - Possono essere implementate anche con funzioni **virtuali pure**
- **classi derivate:** classi più specifiche che **ereditano** le caratteristiche comuni della classe base e le estende

usi: quando si devono implementare molte classi con poche operazioni (*specifiche per la classe)*

*es. classe **shape***

```
class Shape{
public:
    void draw() const;
   virtual void move(int dx, int dy);
   ....
  virtual ~Shape(){};

protected:
   Shape()
  Shape(initializer_list<Point> lst);
  virtual void draw_lines() const;
 .....

private:
  vector<Point> points;
  ....
}
```

protected = funzioni accessibili dalle classi derivate, ma non da altre

- ➔ contiene costruttori (in Shape → scelta **progettuale**) perché utenti non possono creare degli oggetti shape (*è una classe che definisce un concetto astratto, se disegno una figura è shape ma meglio definita come qualcos'altro → es Circle*)

private = funzioni **non** accessibili nemmeno nelle classi derivate

es. classe derivata → ***Circle***

```
class Circle : public Shape {
....
}
```

eredita **TUTTI** i membri (public, protected, private) MA può accedere solo a quelli public e protected

Is a

> = rapporto tra una classe base e la sua derivata (**legame logico**)
> *"un oggetto della classe derivata è un [**is a**] oggetto di classe base"*

ATTENZIONE: non è biunivoca (un oggetto della classe base non è un oggetto di classe derivata)

PROBLEMA DELLO SLICING

- avviene quando si tenta di inserire una classe derivata in uno slot che riesce a contenere **solo** membri della classe base

→ compilatore risolve affettando l'oggetto → lasciando solo i membri che "ci stanno"

```
void my_fct(const Circle& c)
{
   vector<Shape> v;
  v.push_back(c)
}
```

probabilmente Circle ha membri aggiuntivi rispetto a Shape (es. raggio) che non riescono ad essere contenuti nel vector

=> disabilito il costruttore e l'assegnamento di copia nella classe base

```
class Shape {
public:
    Shape(const Shape&) = delete;
    Shape& operator=(const Shape&) = delete;
```

has a

definisce la composizione tra classi (un oggetto di una classe contenuto in un'altra classe)

[11.2]

Funzioni virtuali

recap gerarchie classi

- ***ereditarietà/derivazione***: *una classe eredita dati e funzioni membro dalla classe base*
- ***funzioni virtuali***: *possibilità di definire la stessa funzione nella classe base e in quella derivata*
    - *polimorfismo run-time pk si decide quale funzione chiamare a tempo di esecuzione*
- ***incapsulamento:*** *membri private e protected per nascondere dettagli implementativi*

<u>overriding:</u> quando una classe derivata **ridefinisce** una funzione virtuale di una classe base => classe derivata sfrutta l'interfaccia della classe base

- funzione oggetto di override ha <u>stesso nome, stessi tipi e stessa constness</u>

<u>es.</u>

```
class Shape {
    virtual void draw_lines() const;
}
class Circle : public Shape {
    void draw_lines() const;
}
```

overriding con i **puntatori**

```
class B{}
class D: public B{}
class DD: public D{}
```

- puntatore a B può puntare ad un oggetto D o DD
- D **is a** B, DD **is a** B e **is a** D
- **vector<*B>** può gestire una **collezione** di oggetti diversi (derivanti da B)

es

```
void call(const B& b){
   b.f()
}
```

→ In call avviene uno sdoppiamento di come appare B

- oggetto **formale**: B
- oggetto **reale**: B, D o DD (quello passato alla chiamata della funzione)

Esistono delle funzioni **espressamente** progettate per l'override

- keyword **override**
- **errore di compilazione** se non implementato
- utile per chiarezza & per sicurezza nella gestione dell'override

```
class Shape {
    virtual void draw_lines() const;
```

```
}
class Circle : public Shape {
    void draw_lines() const override;
}
```

errori:

- se la funzione nella classe base non è **virtual**
- se ha firma diversa

funzioni virtuali pure: esplicitamente non possono essere implementate nella classe base
→ rendono la classe **virtuale pure =>** impossibile istanziare oggetti di questa classe

```
class B{
public:
    virtual void f() = 0;
}
```

<u>scopo</u>

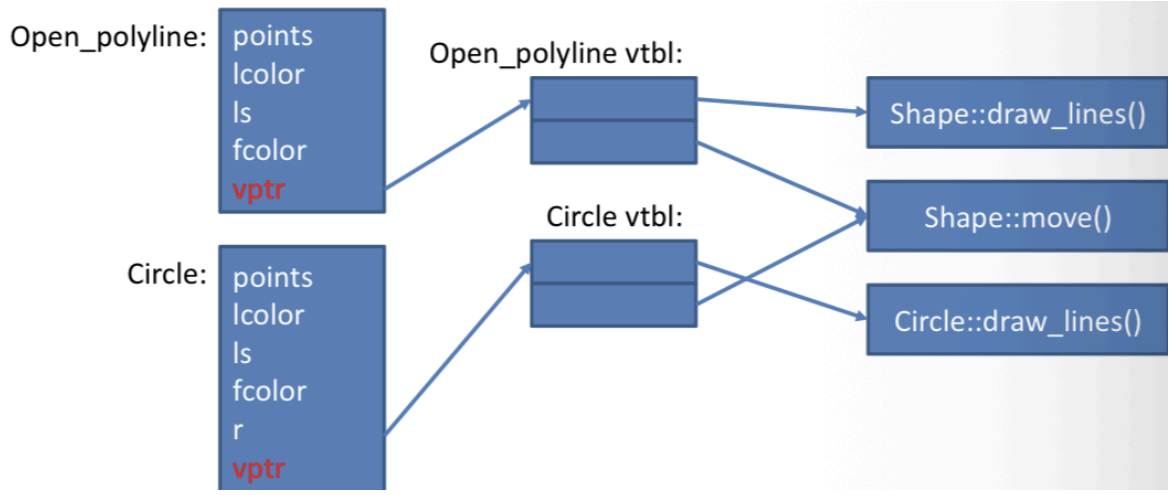
- impedire di istanziare oggetti di questa classe
- **obbligare** tutte le classi derivate a implementare una determinata funzione

[12.1]

LAYOUT DEGLI OGGETTI

Consideriamo la classe SHAPE e due sue classi derivate (Open_line e Open_polyline)
=> I loro dati membro sono aggiunti dopo quelli della classe base

Virtual table (vtbl)
una chiamata a funzione **virtuale** è gestita tramite le vtbl
—> sono gestire tramite **vptr**
Per ogni classe (**non oggetto**) è definita una vtbl che definisce quali funzioni devono essere chiamate



Per ogni **oggetto** è definito un puntatore virtuale alla virtual table della sua classe

La virtual table contiene ulteriori **puntatori**

- un puntatore per ogni funzione della propria classe
- Se una funzione è virtuale → usa il **virtual pointer** e grazie ad esso risolve la chiamata

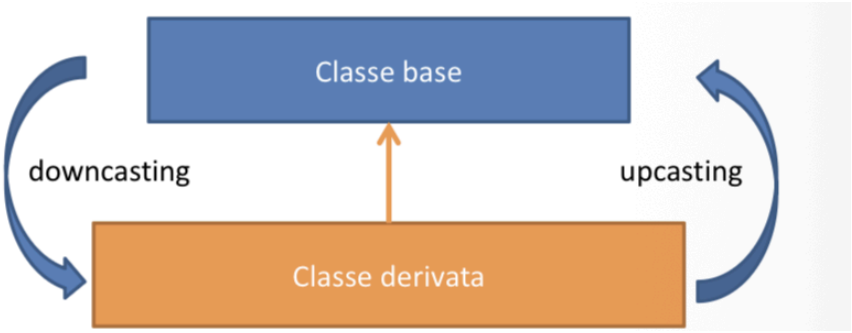
C'è un **overhead** dovuto al virtual pointer, mentre il numero di funzioni virtuali ha un impatto sulla dimensione delle virtual table
=> l'overhead principale è la **dereference**

[12.2]

Casting

Dynamic cast

Per "navigare" tra le classi



Upcasting: sempre lecito
-> attraverso **is a** non è necessario un cast poiché un oggetto di classe derivata è un oggetto di classe basse

Downcasting: con puntatori e reference.

- effettuato con **dynamic_cast**
- funziona con **puntatori e reference**

- Effettuato con **verifiche a run-time** => non è sempre lecito. ATTENZIONE: controllare sempre che vada a buon fine altrimenti:
    - puntatore ritorna null
    - reference: eccezione

[12.3]

Standard Template Library

Per gestire le situazioni in cui si hanno grandi **quantità di dati** - >so delle **strutture** di dati che permettono di effettuare operazioni sui dati in modo semplice
Alcune operazioni vengono effettuate **indipendentemente** a che struttura dati contiene i dati -> a livello di algoritmo sono simili

> **STL:** una parte della libreria standard che contiene
> - **contenitori** di dati
> - **algoritmi generici** (per effettuare le operazioni frequenti con implementazione indipendente da dove sono contenuti i dati)

E' necessario aggiungere codice aggiuntivo solo se si vuole implementare funzioni aggiuntive

Contenitori

(13.5): sequenza di elementi che supporta le operazioni di copia e chiama il tipo di loro elementi value_type

I contenitori STL hanno accesso **unificato:** modificare/accedere ad un contenitore piuttosto che un altro non cambia (non vengono usate funzioni differenti per ogni contenitore)

- Se voglio cambiare contenitore non devo cambiare metodi di accesso/modifica
- accesso indipendente da come sono salvati i dati e dal tipo usato

Es.

- Linked list
- std::vector
- std::string
- std::list

sequenza
Ha inizio e fine ed è gestita tramite **iteratori** (passo agli elementi successivi con ++)

iteratori = oggetto che punta ad un elemento specifico di una sequenza
- Iteratore **begin:** punta al primo elemento
- Iteratore **end:** primo elemento **dopo** l'ultimo

E' una collezione di classi. E' definita per ogni tipo di struttura
- Posso confrontare due iteratori

È possibile riferirsi agli oggetti puntati usando l'operatore unario ***** (dereference)

keyword auto: il tipo dell'iteratore viene dedotto dal compilatore (uso con iteratori, non con altro)

Disaccoppiamento dati/algoritmi
E' necessario separare gli elementi
- Collezione di dati
- Struttura di dati

Interagiscono molto tra loro, ma è utile disaccoppiarli (*posso cambiare struttura dati senza cambiare funzioni, uso sempre le stesse per diverse strutture*)

Disaccoppiamento lato algoritmo	non è necessario conoscere la struttura dati e i suoi dettagli per poter lavorare sui dati contenuti in esse. Sufficiente usare i relativi **iteratori**
Disaccoppiamento lato struttura dati	non espongo le caratteristiche della struttura dati e gestire tutti i modi d'uso, ma è.necessario solo implementare ed esporre gli **iteratori**

```
// Restituisce l'elemento in [first, last) che ha
// il valore più alto

template<typename Iterator>
Iterator high(Iterator first, Iterator last)
{
    Iterator high = first;
    for (Iterator p = first; p != last; ++p)
        if (*high < *p)
            high = p;

    return high;
}
```

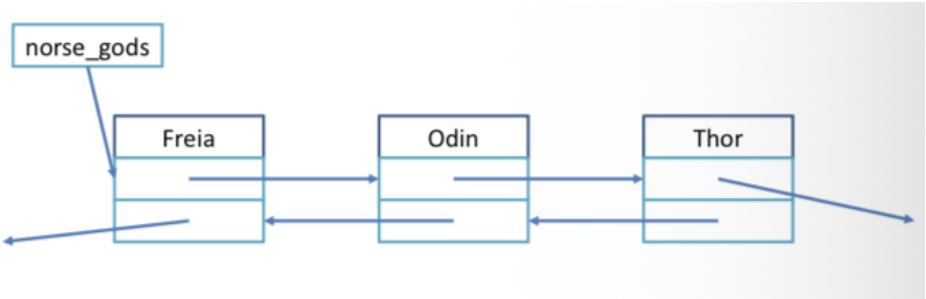
Osservazione: è un'operazione che è indipendente dalla struttura usata per contenere i dati

- *é unbounded: non noto una classe che obbliga un certo comportamento*

(13.4)

Linked List

struttura gestita come



Implementazione

```
template<typename Elem>
struct Link {
    Link* prev;
    Link* succ;
    Elem val;
};

template<typename Elem> struct list {
    Link<Elem>* first;
    Link<Elem>* last;
};
```

Operazioni che coinvolgono una lista

- Quelle di vector (no /parentesi quadre)
- Insert ed erase (eliminare e inserire nodi)
- Iteratori: pk è una STL

```
template<typename Elem>
class list {
    // rappresentazione e dettagli implementativi
public:
    class iterator;

    iterator begin();
    iterator end();

    iterator insert(iterator p, const Elem& v);
    iterator erase(iterator p);

    void push_back(const Elem& v);
    void push_front(const Elem& v);
    void pop_front();
    void pop_back();

    Elem& front();
    Elem& back();
    // ...
};
```

Restituiscono un iteratore!

```
template<typename Elem>
class list<Elem>::iterator {
    Link<Elem>* curr;
public:
    iterator(Link<Elem>* p) : curr{p} {}
    iterator& operator++() { curr = curr->succ;
                             return *this; }
    iterator& operator--() { curr = curr->prev;
                             return *this; }
    Elem& operator*() { return curr->val; }

    bool operator==(const iterator& b) const {
                        return curr == b.curr; }
    bool operator!=(const iterator& b) const {
                        return curr != b.curr; }
};
```

Begin == end —> lista vuota
end punta all'elemento DOPO la fine, cosi non devo trattare il caso lista vuota come caso speciale

problema: difficile gestire l'iteratore e di che tipo è => il compilatore deduce di che tipo è attraverso la keyword **"auto"** che avviene durante l'inizializzazione dell'iteratore

```
template<typename T>
void user(vector<T>& v, list<T>& lst)
{
    for (auto p = v.begin(); p != v.end();
        ++p) cout << *p << '\n';

    auto q = find(lst.begin();
        lst.end(); T{42});
}
```

ATTENZIONE: NEL CORSO ACCETTABILE SOLO PER SCORRERE ALL'INTERNO DEI FOR
Perché molte volte può rendere il codice ambiguo

(13.5)
Std::vector

- supporta tutte le operazioni (insert(), erase() anche se inefficienti perché muovono molti elementi in memoria)
- fornisce/parentesi quadre
- Range check
- espandibile
- operatore ==

Std::string

- operazioni manipolazione del testo
- operatore ==
- è diverso da char/quadre
    - non conosce la propria dimensione
    - non ha begin(), end() e altri operatori
    - no ange check
    - elementi non contigui in memoria
    - ==, != e output si riferiscono al puntatore al primo elemento

Std::list

- fornisce le solite operazioni tranne /parentesi quadre
- insert e erase senza spostare gli elementi
- espandibile
- operatore ==

[14.1]
std::map

> contenitore ordinato di coppie chiave-valore, ottimizzato per la ricerca di chiavi

Header: <map>

std::set

> map senza i valori. Contenitore ordinato attraverso una funzione d'ordine fornita per ciascun tipo contenuto

Header: <set>

- non è gestito con []
- non viene usato push_back()
- gestito con **insert()** e **erase()**

[14.1]

Algoritmi STL

La Standard Template Library contiene una collezione di algoritmi utilizzabile dal programmatore (deve solo includerli → **#include <algorithm>**)

Sono compatibili con tutti i container stl

Find()

= cerca un elemento con un dato valore in una sequenza definita tramite gli iteratori [first, last]
→ generico rispetto al tipo di container e tipo contenuto

ritorna: iteratore alla prima occorrenza trovata oppure end() se il valore non è stato trovato

Find_if()

verifica che un **criterio** sia verificato.
predicato: è come viene espresso un criterio. E' una funzione che ritorna true o false

```
bool odd(int x){ return x%2; }
void f(vector<int>& v){
   auto p = find_if(v.begin, v.end. odd); //odd → predicato
}
```

Nome di una funzione senza () è il **puntatore alla funzione**
Se devo passare un valore alla funzione usata come predicato → NO, argomento, OK variabile come variabile globale
problemi problemi 🚗: il valore della variabile globale è visibile e modificabile a tutti

alternativa : function object
= oggetto usato come una funzione.
Possibile usarla come predicato aggiungendo l'overloading dell'operatore ()

```
class Larger_than {
   int v;
public:
   Larger_than(int vv) : v{vv}{}
   bool operator() (int x) const {return x>y}
 }

void f(list<double>& v, int x){
 auto p = find_if(v.begin(), v.end(), Larger_than(x);
}
```

1. Function object **creato** nell'argomento
2. oggetto **passato** a find_if() **per copia** (se oggetti grandi → meglio riferimento)
3. find_if chiama l'operator()

[14.3]
alternativa a function object: **lambda expression**, cioè un modo per definire una funzione localmente. (differenza: function object viene creato altrove mentre lambda expression non è utilizzabile altrove)

- cattura le variabili locali per riferimento
- non è definito il nome

Sort()
=effettua un ordinamento tra due iteratori (→ posso ordinare una parte del container)

- con operator <: *es. void sort(Ran first, Ran last);*
- con funzione dedicata: es. *void sort(Ran first, Ran last, Cmp cmp);*
    - *Cmp è un **comparison function Object** → deve essere comparable e accettare due argomenti a e b e ritornare true se a compare prima di b*

Per ordinare un intero container uso la funzione sort() della libreria **std_lib_facilities.h**

Per ordinare oggetti con più dati membro → uso predicato per stabilire quale confrontare

- **function object** con operator() per gestire il confronto tra i dati membro
- **lambda expression**

Copy()

= effettua una copia di una sequenza in un'altra sequenza (anche su container diversi)
ATTENZIONE: è l'utente che deve verificare che lo spazio sia sufficiente

es. copy(li.begin(), li.end(), vd.begin());

- *inizio e fine della sorgente*
- *inizio della destinazione*

Copy_if()

= copia con la verifica di un **predicato**
*es. copy_if(v.begin(), v.end(), v2.begin(), **Larger_than(6)**);*

[lab 9_1]

CMake

Progetti: composto da più file sorgente (.h .cpp .hpp) e possono avere piu file eseguibili/librerie
Descrivere progetti necessita

- file sorgenti
- sapere come e' organizzata la compilazione
    - **target di compilazione (***creare un eseguibile/libreria → lista dei .cpp e delle librerie linkate)*

Descrizione tramite comando di compilazione: serve riscrivere tutto ad ogni compilazione
Descrizione tramite progetto dell' IDE: dipende dall' IDE

sistema di building standard

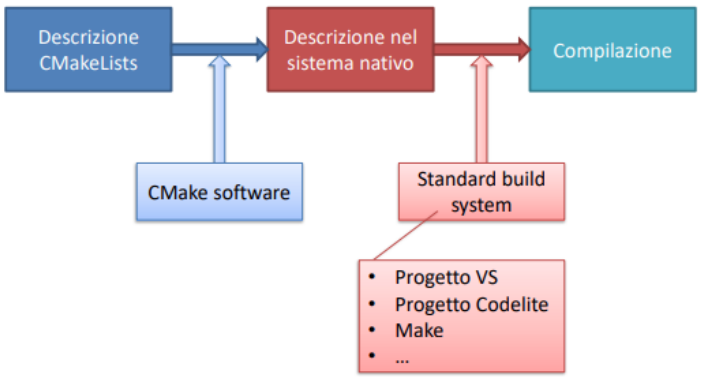
=gestore del processo di compilazione

CMake: meta sistema di building standardizzato indipendente da

- compilatore
- piattaforma

usato in abbinata con il sistema di building nativo
scopo: distribuire codice sorgente con indicazione di building

definisce i target di compilazione + librerie da linkare
Gestito con file di testo: CMakeLists.txt



LATO UTENTE

Scaricare un sorgente gestito con CMake permette di avere:

- sorgente
- La **descrizione** del progetto
- Un sistema che predispone la **compilazione**
- Un sistema per gestire la **compilazione selettiva** dei moduli (può essere necessario compilare solo alcune parti)

fasi

1. **configurazione**
    a. creazione directory build
    b. configurazione tramite comando cmake
2. **compilazione:**
    a. comando make

```
user@host:~/Rational/$ mkdir build
user@host:~/Rational/$ cd build
user@host:~/Rational/build/$ cmake ..
user@host:~/Rational/build/$ make
```

Usare la directory **build** come output è molto utile:

- Non mescola output di compilazione con il progetto sorgente
- Permette di ritornare alla situazione iniziale eliminando la directory

LATO SVILUPPATORE

creazione della descrizione

- scrivere a mano
- usare IDE automaticamente

ATTENZIONE: una volta scelto un metodo → usare sempre quello altrimenti sovrascrivo parti (*casotto)*

come?

1. creare un **CMakeList.txt**
    a. scelgo una **versione** (per noi va bene una vecchia per le nostre necessita', non e' necessario avere la stessa versione dell'IDE)
2. settare lo **standard di compilazione**
3. **nominare** un progetto → implicitamente sto settando una variabile d' ambiente
4. impostare le **librerie esterne** (*solitamente noi non la mettiamo)*
5. inserire le **directory** dove cercare il file **header** (rispetto a CMakeList.txt → basta specificare il loro nome)
6. **add_library**: crea la libreria dinamica (path relativo del sorgente associato)
7. **nome al target** → lancia l'eseguibile main alla libreria librational (un certo eseguibile deve essere linkato con una certa libreria)
8. installazione nel **sistema della libreria e degli header**
9.

```
cmake_minimum_required(VERSION 2.84)
set(CMAKE_CXX_STANDARD 11)
project(rational)

find_package(Qt5 REQUIRED COMPONENTS Core)

include_directories(include)

add_library(${PROJECT_NAME} SHARED
                        src/rational.cpp)

add_executable(main src/main.cpp)

target_link_libraries(main ${PROJECT_NAME})

install(TARGETS ${PROJECT_NAME}
         LIBRARY DESTINATION lib
                 INCLUDES DESTINATION include
)
```

```
cmake_minimum_required(VERSION 2.84)
set(CMAKE_CXX_STANDARD 11)
project(myvector)

add_library(${PROJECT_NAME} INTERFACE)

target_include_directories(${PROJECT_NAME} INTERFACE
          $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
          $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/src>
)


add_executable(main src/main.cpp)
target_link_libraries(main ${PROJECT_NAME})
```

(13.1)

Eccezione

Libreria standard possiede le **standard exception —>** classi che danno una buona descrizione delle eccezioni che vengono lanciate

- Std::logic_error: errori logici come non raggiungere gli invarianti
  - out_of_range
- std::runtime_error: esterni al programma stesso (difficilmente prevedibili)
  - di input output

```
class exception {
        public:
                exception();
                exception(const exception&);
                exception& operator=(const exception&);
                virtual ~exception();
                virtual const char* what() const;
};
```

Osservazioni:

- *What() restituisce una stringa di messaggio*
- *costruttore e assegnamento di copi*

```
try {
        // ...
        // ...
} catch (std::exception& e) {
        std::cerr << e.what() << std::endl;
}
```

Tipi di eccezioni

- Eccezioni dedicate: classi create da noi
- Eccezioni standard
- Eccezioni derivate di std::exception —> creiamo noi una classe derivata creando il nostro what()

ATTENZIONE: scegliamo noi quale sia la classe più opportuna e al livello più consono

Risorse ed eccezioni

Es. La gestione delle risorse può portare a eccezioni ne compromettono il funzionamento

esempio: vector accede alla memoria con due metodi (at() e operator /parentesi quadre
come gestisco? ***Eccezioni***

ATTENZIONE: usarle nel modo scorretto potrebbero compromettere il codice.

```
void suspicious(int s, int x)
{
    int* p = new int[s];
    // ...
    delete[] p;
}
```

Se nei … metto <u>solamente</u> una eccezione => non viene eseguito il delete perché chiamo l'eccezione ed esco subito

```
void suspicious(int s, int x)
{
    int* p = new int[s];
    vector<int> v;
    // ...
    if (x) p[x] = v.at(x);    // at(): accesso con verifica
    // ...                    // può lanciare eccezioni
    delete[] p;
}
```

È un problema di risorse o eccezione?
-> mix di entrambe
soluzione

```
void suspicious(int s, int x)
{
    int* p = new int[s];
    vector<int> v;
    // ...
    try {
        if (x) p[x] = v.at(x);
        // ...
    } catch (...) {
        delete[] p;
        throw;        // rilancia l'eccezione
    }
    // ...
    delete[] p;
}
```

Interrompo la gestione dell'eccezione temporaneamente —> gestisco le risorse —> rilancio

Ma troppo lungo da fare ad ogni lettura di vettori (molto frequente -> ad ogni assegnamento e allocazione)

altra soluzione: **Resource Acquisition Is Initialization (RAII)**
uso std::vector al posto dell'allocazione dinamica… perché?

Memoria acquisita in fase di costruzione è liberata dal distruttore in fase di distruzione -> quando il flusso lascia uno scope sono **invocati TUTTI i distruttori degli oggetti**
=> risolve i problemi di combinazione tra eccezioni e memoria e di design

Problema:

- *Alcune classi non gestiscono il funzionamento standard*
- *A volte il distruttore non segue le nostre necessità (es. devo uscire dallo scope senza liberare le risorse)*

esempio soluzione un pò meh

```
vector<int>* make_vec()
{
    vector<int>* p = new vector<int>;
        // ... riempimento del vettore con i dati - può
        // ritornare un'eccezione!
    return p;
}
```

CATTIVA PRATICA: *ma per ora ci da l'idea della necessità di "portarsi fuori" gli oggetti dallo scope*
problema:

- *new chiama il costruttore e alloca la memoria ma non viene chiamato il distruttore (delete) -> non viene lasciata la memoria =>* ***gestione sbagliata delle risorse .*** *tuttavia questo bypass della gestione standard della memoria permetterebbe di fare che ci serve.* Ci DOBBIAMO assicurare che venga liberata la memoria dal chiamante
- Se dobbiamo gestire le eccezioni si complica tutto

Garanzie i gestione risorse && eccezioni

- **BASIC Guarantee:** blocco tra/catch (come es sopra di make_vec) -> gestisce i casi base che danno problemi
    - STL le fornisce
- **strong Guarantee:** la funzione rispetta le basic Guarantee e tutte i valori (non locali alle funzioni) rimangono gli stessi che erano presenti prima della chiamata in fallimento
- **no-throw guarantee:** la funzione non lancia eccezioni

Su make_vec c'è modo per evitare il blocco tra/catch?
=> collegando uscita dallo scope e deallocazione => **Smart pointer**

(13.3)

Smart Pointer

=classi template contenute nell'header <**memory**> che gestiscono la deallocazione della memoria quando escono dallo scope. Eliminano

- Memory leak
- deadling pointer

Tipi che differiscono per caratteristiche fondamentali:

- Unique_ptr
- share_ptr

Si può dire che lo smart pointer detiene un puntatore -> lo sta "proteggendo"

unique_ptr

- Dealloca la memoria uscendo dallo scope
- permette i move
- **non permette le copie** (con = delete, o mettendolo private)
- non ha overhead rispetto ad un puntatore

FASI

- inizializzazione:
  - Nel costruttore
  - funzione reset = distrugge un oggetto puntato se esiste e inserisce un nuovo puntatore nell'unique. È una sorta di deallocazione
- Accesso: operatori * e ->
- Liberazione automatica della memoria: se viene distrutto unique viene distrutto anche l'oggetto
- Rilascio del puntatore:
  - realise() estrae il puntatore dallo unique_ptr => resettato a nullptr

Esempio

```
vector<int>* make_vec()
{
    unique_ptr<vector<int>> p { new vector<int> };
       // ... riempimento del vettore con i dati - può
       // lanciare un'eccezione!
    return p.release();
}
```

Release() perde la deallocazione automatica
osservazione: allocare dinamicamente uno std::vector è ***dannoso*** *(fatto solo per esempio)*

Ora ***ritornare uno unique_ptr*** *è OK! Perché non usa la copia ma il move*
=> posso usarlo per risolvere la necessità di gestire la delete nel chiamante

ATTENZIONE: non si possono usare due unique_ptr sullo stesso oggetto.

- quando faccio la deallocazione del contenuto di uno, anche il contenuto del secondo è stato allocato perché è lo stesso => doppia cancellazione + deadling pointer

Share_ptr

=rimuove i vincoli di unique_ptr

- Può essere **copiato**

- essere **condiviso**

Ha il **reference counting:** shared_ptr che hanno lo stesso puntatore sanno della loro esistenza reciproca = > elimina il problema della doppia deallocazione perché solo **l'ultimo** esistente effettua la **distruzione**